

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(UGC-Autonomous Institution), Ghanpur (V), Ghatkesar (M), Medchal (D), Hyderabad - 500088

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

CHAPTER 5: SYSTEM DESIGN

ROLE BASED MULTI AGENT LLMS FOR TRANSPARENT DECISION MAKING

Review 03 Report

Under the guidance of: Dr. Pulime Satyanarayana, Professor, Dept. of CSE (AI&ML)

Presented by: Batch 10

N. Vishnu Kumar (23RA1A66D4) | G. Vinay Goud (23RA1A66D1)

M. Yeshwanth (23RA1A66E0) | K. Shiva Sai (23RA1A66A5)

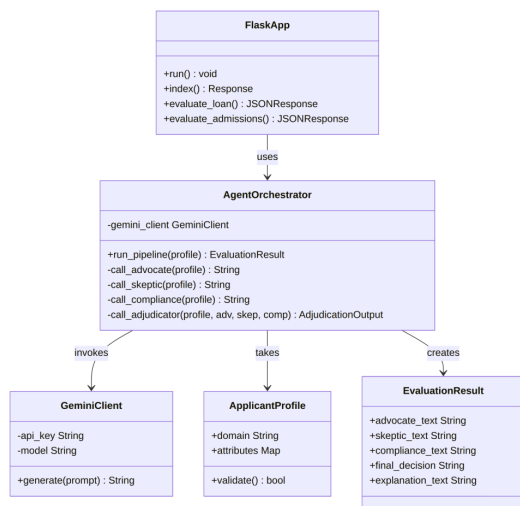
5. System Design

This chapter presents the design of a web-based decision support system in which four role-based LLM agents deliberate over an applicant's case and produce a transparent, explainable decision. A Flask application receives the applicant data from a human reviewer and passes it to an orchestration layer. The orchestrator runs the Advocate, Skeptic, Compliance and Adjudicator agents in order, all powered by the Google Gemini API. The first three agents each produce a brief, and the Adjudicator uses the applicant profile and those briefs to return the final decision. Every intermediate output is shown as a deliberation card, which makes the decision transparent.

The design is described through six UML diagrams (class, use case, sequence, component, deployment and activity) and two Data Flow Diagrams (Level 0 and Level 1). Each is summarised below with its key points.

5.1 Class Diagram

Shows the main classes, their responsibilities and relationships (static structure).



- **FlaskApp** handles HTTP requests and delegates to AgentOrchestrator.
- **AgentOrchestrator** validates data via ApplicantProfile and runs the four agent stages.
- **GeminiClient** calls the Gemini API; no other class talks to it directly.
- **EvaluationResult** holds the agent outputs, final decision and explanation.

Figure 5.1: Class diagram

5.2 Use Case Diagram

Shows who uses the system and which functions each actor performs.

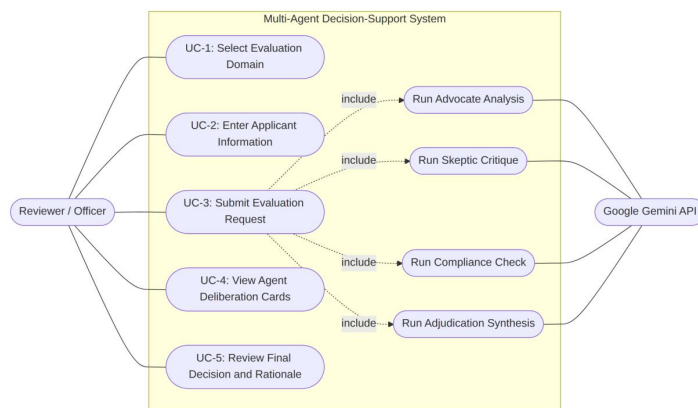


Figure 5.2: Use case diagram

- **Actor:** Reviewer / Officer.
- Selects the domain, enters applicant information and submits the request.
- Views the agent deliberation cards and reviews the final decision and rationale.
- **Google Gemini API** runs advocate, skeptic, compliance and adjudication.

5.3 Sequence Diagram

Shows the order of messages exchanged during one evaluation (dynamic behaviour).

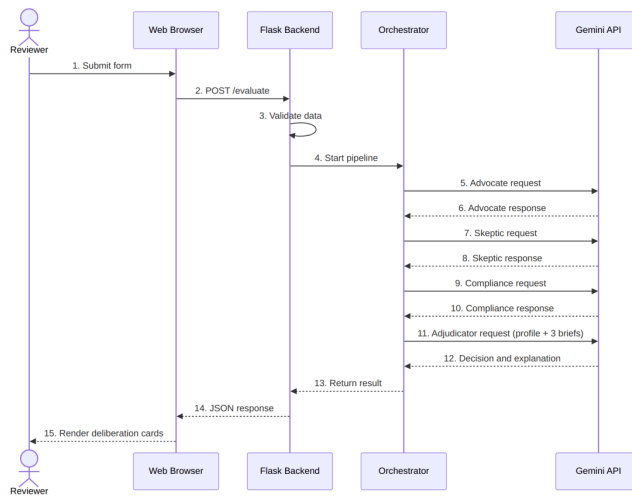


Figure 5.3: Sequence diagram

- Reviewer submits the form; **POST /evaluate**; Flask validates the data.
- Orchestrator calls Gemini in order: **Advocate, Skeptic, Compliance**.
- Adjudicator receives the profile and 3 briefs and returns the decision.
- JSON response is rendered as deliberation cards.

5.4 Component Diagram

Divides the system into replaceable modules and shows their dependencies.

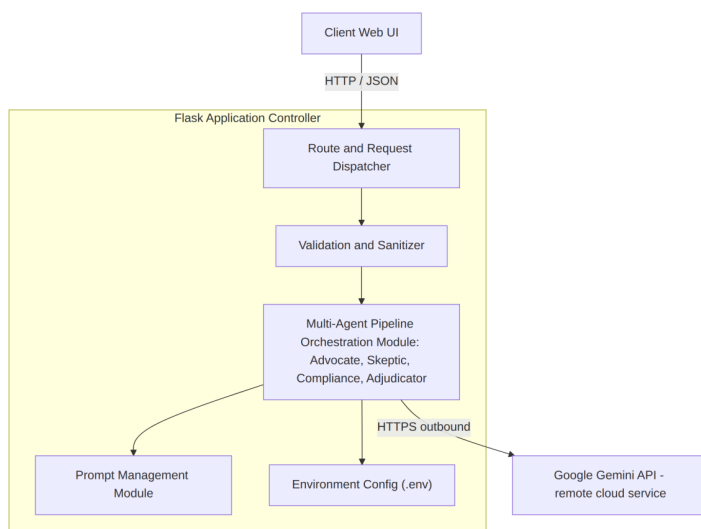
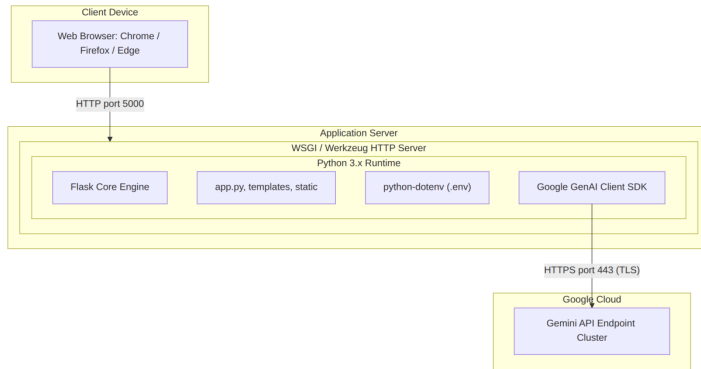


Figure 5.4: Component diagram

- Route dispatcher, validation and sanitizer.
- Multi-agent orchestration module.
- Prompt management module and environment config (**.env**).
- Design principle: **high cohesion and loose coupling**.

5.5 Deployment Diagram

Shows where the software runs and how the nodes communicate.

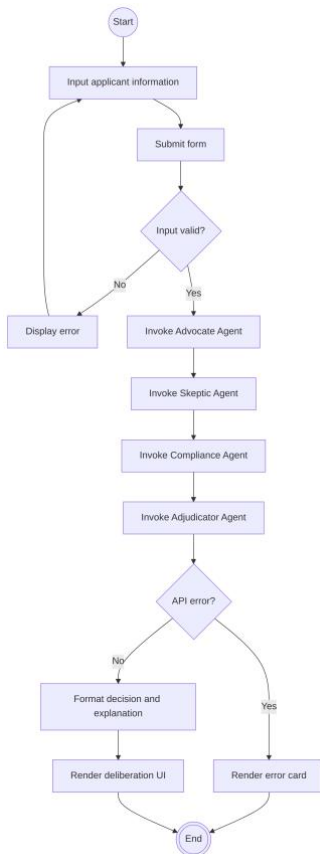


- Browser connects to the Flask server over **HTTP, port 5000**.
- Server runs Python 3.x with Flask, python-dotenv and the GenAI client SDK.
- Server connects to the Gemini API on Google Cloud over **HTTPS, port 443**.

Figure 5.5: Deployment diagram

5.6 Activity Diagram

Models the flow of control from form submission to the displayed result.



- Input applicant information and submit the form.
- If the input is **invalid**, show an error and return to input.
- If **valid**, invoke Advocate, Skeptic, Compliance, Adjudicator in order.
- On an API error show an error card; otherwise render the deliberation UI.

Figure 5.6: Activity diagram

5.7 Data Flow Diagrams

5.7.1 DFD Level 0 (Context Diagram)

The whole system as one process with its external entities.

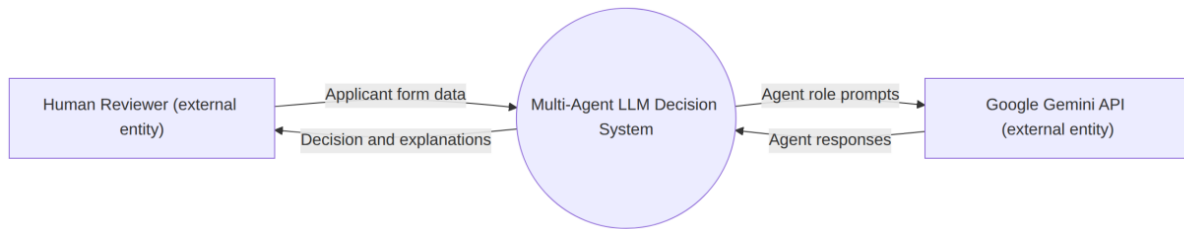
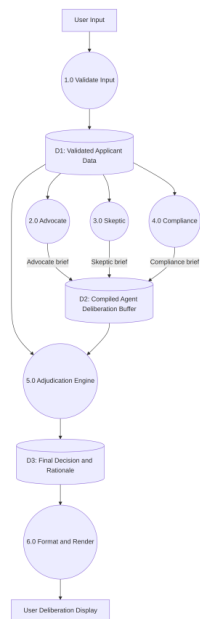


Figure 5.7: DFD Level 0

- External entities: **Human Reviewer** and **Google Gemini API**.
- Reviewer sends applicant form data and receives the decision and explanations.
- System sends agent role prompts to Gemini and receives agent responses.

5.7.2 DFD Level 1 (Decomposed Process)

Breaks the system into sub-processes and data stores.



- **1.0 Validate Input** stores data in D1.
- **2.0 Advocate, 3.0 Skeptic, 4.0 Compliance** produce briefs stored in D2.
- **5.0 Adjudication Engine** stores the result in D3.
- **6.0 Format and Render** shows the output to the user.

Figure 5.8: DFD Level 1

5.8 Summary

The design is modular and loosely coupled: Flask, the orchestrator, the prompt manager and the Gemini client each have one clear responsibility. Every request passes through validation, three agent briefs and a final adjudication, and each agent's output is displayed separately, so the reviewer can inspect how the decision was reached.